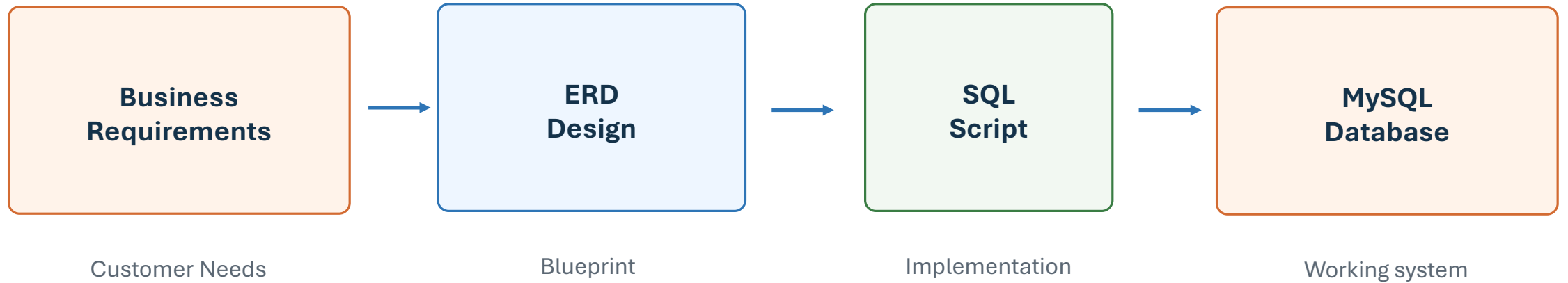


Writing SQL Using DDL, DML, and DQL

Where SQL Fits in the Database Process

SQL is where the database design becomes something MySQL can build and run.



What is an SQL Script?

An **SQL script** is a text file or block of text that contains **one or more SQL statements** that can be executed in sequence by a database system such as MySQL.

An SQL script can contain commands for:

1. **Creating or modifying database structures** — CREATE, ALTER, DROP
2. **Adding or changing data** — INSERT, UPDATE, DELETE
3. **Retrieving data** — SELECT
4. **Creating constraints, views, procedures, triggers**, and other database objects

SQL scripts are commonly saved as files with the **.sql** extension.

An **SQL statement** is a single complete instruction given to a database.

For example:

```
SELECT * FROM Student;
```

That is one SQL statement because it tells MySQL to retrieve all rows from the Student table.

Common SQL syntax rules include:

1. SQL statements are made from keywords such as SELECT, FROM, WHERE, CREATE, and INSERT.
2. Keywords must appear in the proper order.
3. Strings with spaces are enclosed in **single** quotation marks: 'Computer Science'.
4. A semicolon ; normally marks the end of a statement.
5. Table and column names must follow the identifier rules of the DBMS.

For **MySQL**, these are the main identifier rules/conventions:

- **Use letters, numbers, _, and \$.** Examples: Student, Student_ID, Course101.
- **Do not normally start an identifier with a number.** Prefer Room101 rather than 101Room.
- **Avoid spaces.** Prefer StudentName or Student_Name instead of Student Name.
- **Avoid special characters** such as -, /, #, @, and punctuation.
- **Do not use reserved SQL keywords** as identifiers. For example, avoid names such as SELECT, FROM, ORDER, TABLE, and PRIMARY.
- Identifiers should be **descriptive and meaningful**. StudentID is better than SID1 or x.
- MySQL generally allows identifiers up to **64 characters** for common objects such as tables and columns.
- If you absolutely need spaces, special characters, or a reserved word, MySQL lets you enclose the identifier in **backticks**:

SQL coding guidelines

These are not always required by MySQL, but they make SQL easier to read and maintain.

For example, MySQL accepts:

```
select studentid,name from student where major='Computer Science';
```

but this is easier to read:

```
SELECT StudentID, Name  
FROM Student  
WHERE Major = 'Computer Science';
```

A good SQL coding style guidelines is to:

1. Write SQL keywords in **uppercase**
2. Use consistent indentation
3. Put major clauses such as SELECT, FROM, and WHERE on separate lines
4. Use meaningful table and column names
5. End each statement with a semicolon

SQL is generally not case-sensitive for keywords. **Table names** can be case-sensitive depending on the operating system and MySQL configuration. On Windows, table names are usually not case-sensitive; on Linux, they often are.

SQL Keywords

SQL keywords are words that have a special meaning in SQL. They are part of the SQL language and tell the database what operation to perform or how a statement should be interpreted.

Examples of common SQL keywords are:

```
SELECT Name  
FROM Student  
WHERE Major = 'Computer Science';
```

They tell MySQL to:

```
SELECT — retrieve data  
FROM — specify the table  
WHERE — specify a condition
```

A **simple SQL script** is a sequence of SQL statements that performs several database tasks in order. For example, a script can create a database, select it for use, create a table, insert data, and then query that data. Here is a basic example in MySQL:

```
-- Create the database
CREATE DATABASE SchoolDB;

-- Select the database
USE SchoolDB;

-- Create a table
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  Major VARCHAR(50)
);

-- Populate the table with data
INSERT INTO Student (StudentID, Name, Major)
VALUES
(1, 'Alice Johnson', 'Computer Science'),
(2, 'Brian Smith', 'Biology'),
(3, 'Carla Davis', 'Business');

-- Perform a simple query
```

The script performs these steps in sequence:

- 1. CREATE DATABASE SchoolDB; creates a new database named SchoolDB.
- 2. USE SchoolDB; tells MySQL that the following statements should be executed in that database.
- 3. CREATE TABLE Student creates a table with three attributes: StudentID, Name, and Major.
- 4. INSERT INTO adds three student records to the table.
- 5. SELECT * FROM Student; retrieves and displays all rows and columns from the Student table.

The expected query result would look something like:

StudentID	Name	Major
1	Alice Johnson	Computer Science
2	Brian Smith	Biology
3	Carla Davis	Business

CREATE keyword creates a new database named SchoolDB.

You can also think of the structure as:

CREATE	DATABASE	SchoolDB	;
Keyword	Object Type	Object Name	End of statement

Part	Meaning
CREATE	SQL keyword used to create a new database object.
DATABASE	Specifies that the object being created is a database.
SchoolDB	The name of the new database.
;	Marks the end of the SQL statement.

The USE keyword in MySQL selects the **default database** for the current session.

After running a statement such as
USE SchoolDB;

unqualified table names in subsequent SQL statements are assumed to belong to SchoolDB until another database is selected.

You can still access tables in other databases by specifying the database name explicitly, such as HospitalDB.Patient.

In this CREATE TABLE statement:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Major VARCHAR(50)  
);
```

The Name and Major columns have no additional constraints. Therefore, by default, MySQL allows them to contain NULL.

each column definition usually consists of three parts:

Column Name + Data Type + Optional Constraint

VARCHAR stores variable-length text. The number 50 specifies the maximum number of characters allowed. The length is mandatory.

A **constraint** is a rule placed on a column or table to control what data is allowed.

In this example:

```
StudentID INT PRIMARY KEY
```

PRIMARY KEY is a constraint. It means that StudentID must uniquely identify each row in the table.

Use single quotes for Strings (VARCHAR)

A primary key automatically has two important properties:

1. It cannot contain duplicate values.
2. It cannot contain NULL.

This statement inserts **three new rows** into the Student table.

```
INSERT INTO Student (StudentID, Name, Major)
VALUES
    (1, 'Alice Johnson', 'Computer Science'),
    (2, 'Brian Smith', 'Biology'),
    (3, 'Carla Davis', 'Business');
```

Here is the breakdown:

- INSERT INTO — tells MySQL that you want to add new data.
- Student — the table receiving the new rows.
- (StudentID, Name, Major) — the columns that will receive values, in that order.
- VALUES — introduces the data to be inserted.
- Each set of parentheses after VALUES represents **one row**.

For example:

```
(1, 'Alice Johnson', 'Computer Science')
```

means:

- StudentID = 1
- Name = 'Alice Johnson'
- Major = 'Computer Science'

The commas between the parenthesized groups allow you to insert **multiple rows with one INSERT statement**. The final semicolon ends the statement.

This statement retrieves **all data from the Student table**.

```
SELECT *  
FROM Student;
```

- SELECT — tells MySQL that you want to retrieve data.
- * — means **all columns**.
- FROM — specifies which table the data should come from.
- Student — the table being queried.
- ; — marks the end of the SQL statement.

So in plain English:

Display all columns and all rows from the Student table.

If the Student table contains StudentID, Name, and Major, then SELECT * displays all three columns for every student.

DROP DATABASE IF EXISTS removes an existing database while avoiding an error if the database is not present.

For our class, DROP DATABASE IF EXISTS is especially useful because students will often **run the same SQL script many times while practicing and debugging.**

For most part, SQL is case-insensitive, but it can vary depending on Operating System and other factors.

Here is a simple MySQL script that creates two related tables using a **Primary Key (PK)** and a **Foreign Key (FK)**.

```
DROP DATABASE IF EXISTS SchoolDB;
CREATE DATABASE SchoolDB;
USE SchoolDB;

-- Create the Department table
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL
);

-- Create the Student table
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    DepartmentID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
);
```

The DepartmentID column in the Student table is a **Foreign Key**. This creates a relationship between the two tables.

A department can have many students, while each student can be associated with one department.

One important point is that the **parent table should normally be created first**. In this example, Department must be created before Student because Student references it with a foreign key.

If you want to find all students who belong to a particular department, you can use a JOIN.

For example, to find all students in the **Computer Science** department:

```
SELECT Student.StudentID,  
       Student.Name,  
       Department.DepartmentName  
FROM Student  
JOIN Department  
  ON Student.DepartmentID = Department.DepartmentID  
WHERE Department.DepartmentName = "Computer Science";
```

This query:

- 1. Joins the Student and Department tables using DepartmentID
- 2. filters the results to only the Computer Science department
- 3. displays the student's ID, name, and department name

You can break it down like this:

Clause	Purpose
SELECT	Specifies which columns to display
FROM Student	Specifies the main table
JOIN Department	Adds the related Department table
ON ...	Specifies how the two tables are related
WHERE ...	Filters the rows returned